Tomsk State University

Laboratory Report

NEO4J and Python

Student: Liu Ye

Group: 932530

Date: October 7, 2026

Contents

1 Introduction and Environment Setup **2**
- 1.1 Python Environment and Driver . . . 2

2 Connecting to Neo4j **2**
- 2.1 Importing the Driver and Connecting . . . 2
- 2.2 Listing the Available Databases . . . 4

3 Creating the Data **5**
- 3.1 Creating the Home Town . . . 5
- 3.2 Creating the Character Nodes . . . 7
- 3.3 Creating the Episode Nodes . . . 8

4 Establishing Links Between Nodes **10**
- 4.1 Family Relationships . . . 10
- 4.2 Home Relationships . . . 12
- 4.3 Episode Relationships . . . 14

5 Reading the Data **16**
- 5.1 Reading a Single Character . . . 16
- 5.2 Reading the Episodes of a Character . . . 16
- 5.3 Reading the Family and the Home . . . 17

6 Updating the Data **18**
- 6.1 Updating the Properties of a Node . . . 18
- 6.2 Updating the Properties of a Relationship . . . 19

7 Deleting the Data **20**

8 Building DataFrames from the Data **20**
- 8.1 Creating a DataFrame from a Query Result . . . 20
- 8.2 Aggregating the DataFrame and Closing the Connection . . . 22

9 Conclusion **22**

1 Introduction and Environment Setup

In this laboratory the Neo4j database is accessed from Python instead of from the Neo4j Browser. The goal is to repeat the standard CRUD operations – creating, reading, updating and deleting data, as well as establishing links between nodes – from a Python program, and to build a pandas DataFrame from the query results.

The data used in this report is the *Forrest Gump* dataset: the main character, his mother, his wife, his son, his army friend and three episodes of his life. These nodes are added on top of the graph that was already imported into Neo4j in the previous laboratories.

1.1 Python Environment and Driver

The work was done inside a dedicated conda environment called `neo4j`, which keeps the database driver separate from the other Python environments on the machine.

```
conda env list
```

```
(neo4j) liuye@debian:~$ conda env list

# conda environments:
#
# * -> active
# + -> frozen
base                  /data/miniconda3
neo4j                 *  /data/miniconda3/envs/neo4j
pytorch2_12_1_gpu        /data/miniconda3/envs/pytorch2_12_1_gpu

(neo4j) liuye@debian:~$ conda list | grep neo4j
# packages in environment at /data/miniconda3/envs/neo4j:
neo4j                     6.2.0           hd3eb1b0_0
neo4j-python-driver       6.2.0           pyhcf101f3_1    conda-forge
(neo4j) liuye@debian:~$
```

Figure 1: The conda environment and the installed Neo4j driver

The output shows the active environment `neo4j` located at `/data/miniconda3/envs/neo4j`, together with the packages installed in it:

- `neo4j` 6.2.0 – the official Neo4j Python driver
- `neo4j-python-driver` 6.2.0 – its binary dependency

2 Connecting to Neo4j

2.1 Importing the Driver and Connecting

The first Python cell imports pandas and the three classes that are used throughout the notebook: `GraphDatabase` creates the driver, `Driver` and `Result` are used as type hints.

```python
import pandas as pd
from neo4j import Driver, GraphDatabase, Result

URI = "bolt://127.0.0.1:7687"
USER = "neo4j"
PASSWORD = "qpalzm123Q!"
DATABASE = "neo4j"

driver: Driver = GraphDatabase.driver(URI, auth=(USER, PASSWORD))

try:
    driver.verify_connectivity()
    print("Neo4j connection successful!")
except Exception as e:
    print("Neo4j connection failed:", e)
```

The connection requires three things: the Bolt address of the server, a login and a password. `GraphDatabase.driver()` only builds the driver object; it does not open a connection immediately. The connection is checked explicitly by `driver.verify_connectivity()`, which is wrapped in a `try/except` block so that a failure is reported as a message instead of stopping the whole notebook.

The address had to be corrected during the experiment. The first attempt used the address `bolt://192.168.1.1:7687`, which is not the machine where the Neo4j server is running, and the connection was refused.



Figure 2: First connection attempt – the address is wrong and the connection is refused

The error message `Errno 111 Connection refused` means that nothing was listening on that address and port. After the address was changed to `bolt://127.0.0.1:7687`, that is to the local machine itself, the driver connected successfully.

```python
import pandas as pd
from neo4j import Driver, GraphDatabase, Result

URI = "bolt://127.0.0.1:7687"
USER = "neo4j"
PASSWORD = "qpalzm123Q!"
DATABASE = "neo4j"

driver: Driver = GraphDatabase.driver(URI, auth=(USER, PASSWORD))

try:
    driver.verify_connectivity()
    print("Neo4j connection successful!")
except Exception as e:
    print("Neo4j connection failed:", e)
```

✓ 0.0s

```
Neo4j connection successful!
```

Figure 3: Successful connection to the local Neo4j server

The message `Neo4j connection successful!` confirms that the Bolt address, the login and the password are all correct.

2.2 Listing the Available Databases

Before writing any data it is useful to see which databases the server exposes.

```
SHOW DATABASES
```

The query is executed inside a session. A session is opened with `driver.session(database=DATABASE` and closed automatically by the `with` statement, which guarantees that the connection is returned to the driver's pool even if an error occurs.

```python
with driver.session(database=DATABASE) as session:
    result: Result = session.run("SHOW DATABASES")

    for record in result:
        print(record["name"], record["currentStatus"])
```

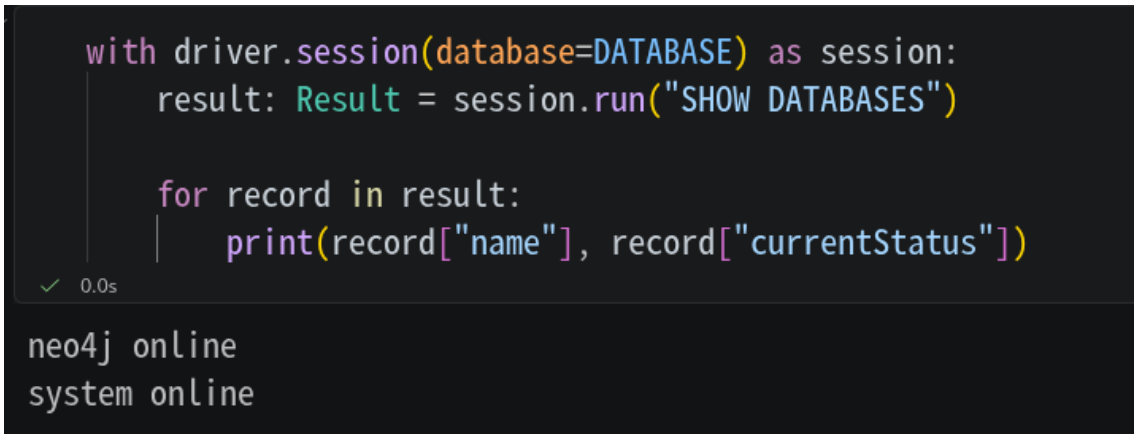


Figure 4: Listing the databases available on the server

Two databases are returned: `neo4j` and `system`. Both have the status `online`. Since the default database `neo4j` is the one holding the imported data, all the following cells pass `database="neo4j"` to the session.

3 Creating the Data

3.1 Creating the Home Town

The first node is the town where the character grows up. It gets the label `City` and two properties: the name and the region.

```
CREATE (city:City {name: 'Greenbow', region: 'Alabama'})
RETURN city
```

The clause `RETURN city` sends the newly created node back to Python, so that the cell can print it and confirm that the node really exists.

```
with driver.session(database=DATABASE) as session:
    result: Result = session.run(
        """
        CREATE (city:City {name: 'Greenbow', region: 'Alabama'})
        RETURN city
        """
    )

    for record in result:
        city = record["city"]
        print(city["name"], city["region"])
```

```python
with driver.session(database=DATABASE) as session:
    result: Result = session.run(
        """
        CREATE (city:City {name: 'Greenbow', region: 'Alabama'})
        RETURN city
        """
    )

    for record in result:
        city: Any = record["city"]
        print(city["name"], city["region"])
```

0.1s

```
Greenbow Alabama
```

Figure 5: Creating the City node for the home town

The output `Greenbow Alabama` shows the two properties of the node that was just created.

The same node can be checked in the Neo4j Browser. The sidebar on the left lists every label and every relationship type that currently exists in the database, and the query below displays the `City` nodes.

```
MATCH (n:City)
RETURN n
LIMIT 25;
```

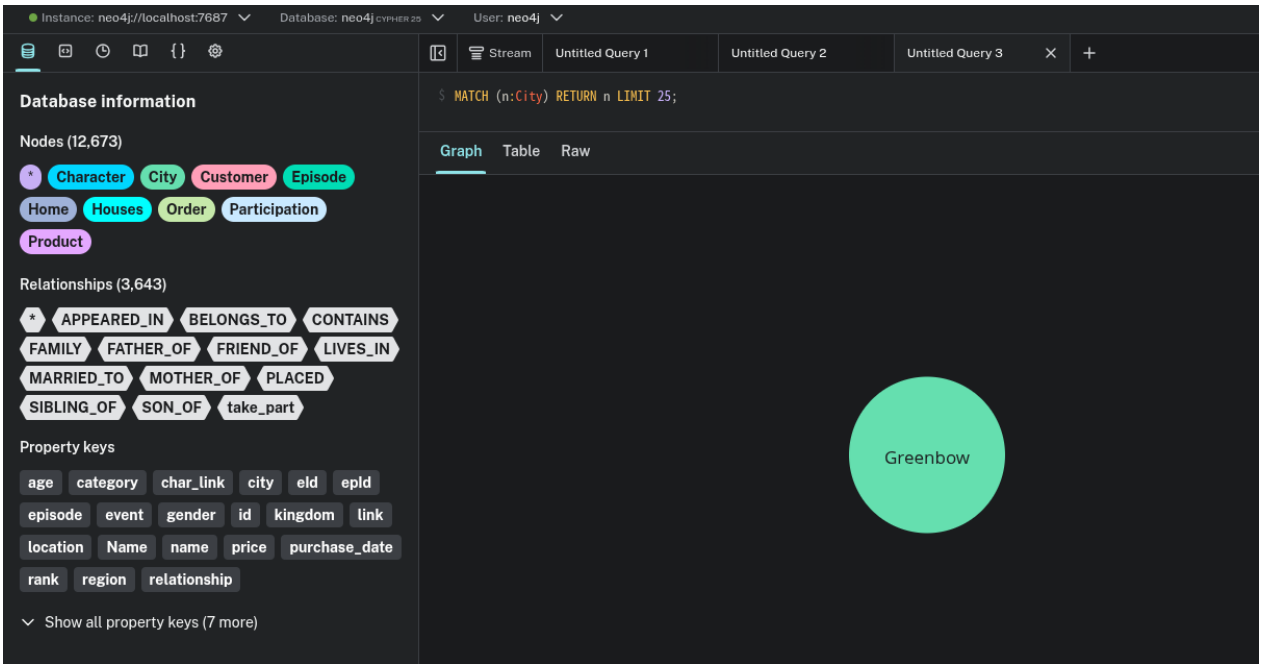


Figure 6: Database overview and the City node in the Neo4j Browser

The database holds 12 673 nodes and 3 643 relationships in total, because it also contains the data imported in the previous laboratories. Among the labels, `Character`, `City` and `Episode` are the three that are used in this report. On the right, the single green node `Greenbow` is displayed the node that was just created from Python.

3.2 Creating the Character Nodes

Five `Character` nodes are created in a single query instead of five separate ones. Each node stores four properties: the name, the age, the state and the role of the person in the story.

```
CREATE
(forrest:Character {name: 'Forrest Gump',     age: 40, state: 'Alabama', role: '
    Hero'}),
(mom:Character     {name: 'Mrs. Gump',        age: 65, state: 'Alabama', role: '
    Mother'}),
(jenny:Character   {name: 'Jenny Curran',     age: 40, state: 'Alabama', role: '
    Wife'}),
(junior:Character  {name: 'Forrest Gump Jr.', age: 7,  state: 'Alabama', role: '
    Son'}),
(bubba:Character   {name: 'Bubba Blue',       age: 40, state: 'Alabama', role: '
    Friend'})
RETURN forrest, mom, jenny, junior, bubba
```

The comma between the node patterns makes `CREATE` treat them as one statement that produces five nodes at once. Every node is given a variable name (`forrest`, `mom`, `jenny`, `junior`, `bubba`) so that the `RETURN` clause can send all of them back.

```
    for record in result:
        for key in ["forrest", "mom", "jenny", "junior", "bubba"]:
            node = record[key]
            print(node["name"], node["age"], node["state"], node["role"])
```

Because the query returns five columns in one single record, the cell uses a nested loop: the outer loop walks over the records and the inner loop walks over the five column names.

```
with driver.session(database=DATABASE) as session:
    result: Result = session.run(
        """
        CREATE
        (forrest:Character {name: 'Forrest Gump',     age: 40, state: 'Alabama', role: 'Hero'}),
        (mom:Character     {name: 'Mrs. Gump',        age: 65, state: 'Alabama', role: 'Mother'}),
        (jenny:Character   {name: 'Jenny Curran',     age: 40, state: 'Alabama', role: 'Wife'}),
        (junior:Character  {name: 'Forrest Gump Jr.', age: 7,  state: 'Alabama', role: 'Son'}),
        (bubba:Character   {name: 'Bubba Blue',       age: 40, state: 'Alabama', role: 'Friend'})
        RETURN forrest, mom, jenny, junior, bubba
        """
    )

    for record in result:
        for key in ["forrest", "mom", "jenny", "junior", "bubba"]:
            node: Any = record[key]
            print(node["name"], node["age"], node["state"], node["role"])
[12]  ✓ 0.0s
...
Forrest Gump 40 Alabama Hero
Mrs. Gump 65 Alabama Mother
Jenny Curran 40 Alabama Wife
Forrest Gump Jr. 7 Alabama Son
Bubba Blue 40 Alabama Friend
```

Figure 7: Creating the five Character nodes

The five printed lines confirm that all the characters were created with the expected properties. The same five nodes are shown below as they appear in the Neo4j Browser.

```
MATCH
(forrest:Character {name: 'Forrest Gump'}),
(mom:Character     {name: 'Mrs. Gump'}),
(jenny:Character   {name: 'Jenny Curran'}),
(junior:Character  {name: 'Forrest Gump Jr.'}),
(bubba:Character   {name: 'Bubba Blue'})
RETURN forrest, mom, jenny, junior, bubba;
```

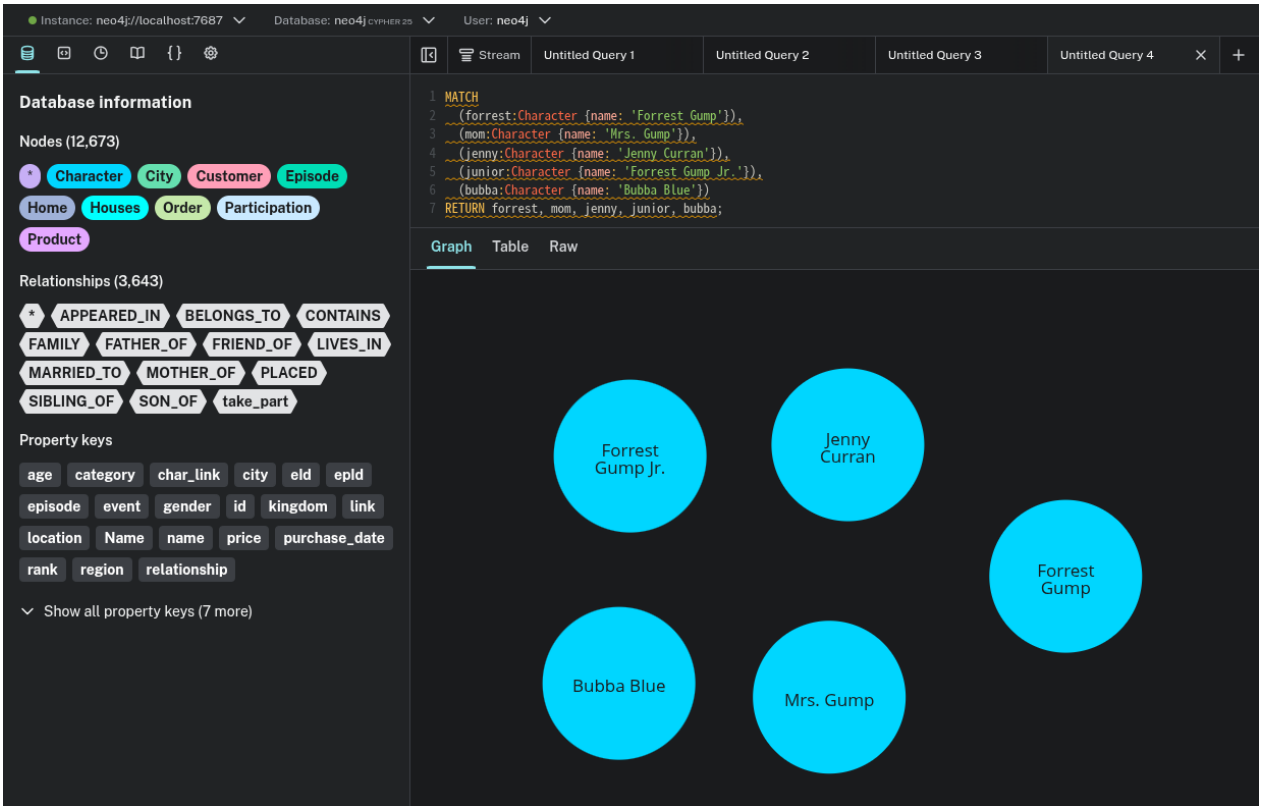


Figure 8: The five Character nodes displayed in the Neo4j Browser

The browser draws five blue circles, one for each character. The colour is chosen automatically according to the label, so every node labelled `Character` is drawn in the same colour.

3.3 Creating the Episode Nodes

In the same way three `Episode` nodes are created. They represent the episodes of the character's life in which he appears. These nodes only carry a title.

```
CREATE
(e1:Episode {title: 'The Vietnam War'}),
(e2:Episode {title: 'Bubba Gump Shrimp Company'}),
(e3:Episode {title: 'Running Across America'})
RETURN e1, e2, e3
```

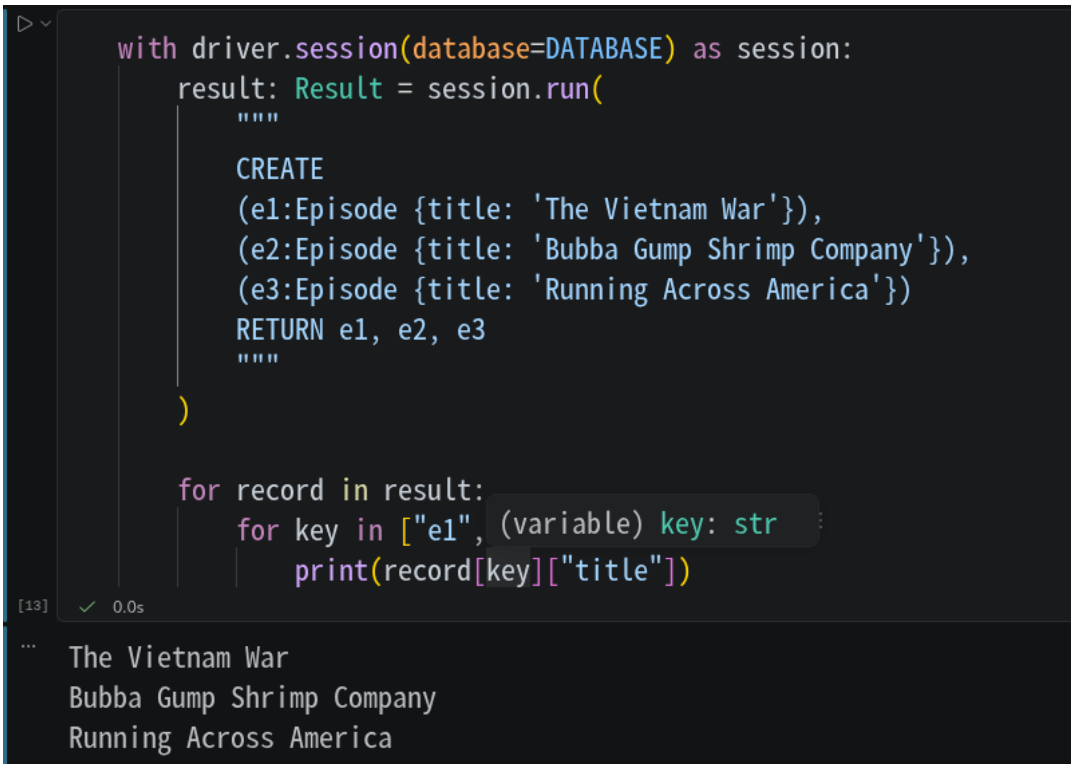


Figure 9: Creating the Episode nodes

The result shows the three episode titles. In the Neo4j Browser the three nodes appear in the same colour.

```
MATCH
(e1:Episode {title: 'The Vietnam War'}),
(e2:Episode {title: 'Bubba Gump Shrimp Company'}),
(e3:Episode {title: 'Running Across America'})
RETURN e1, e2, e3;
```

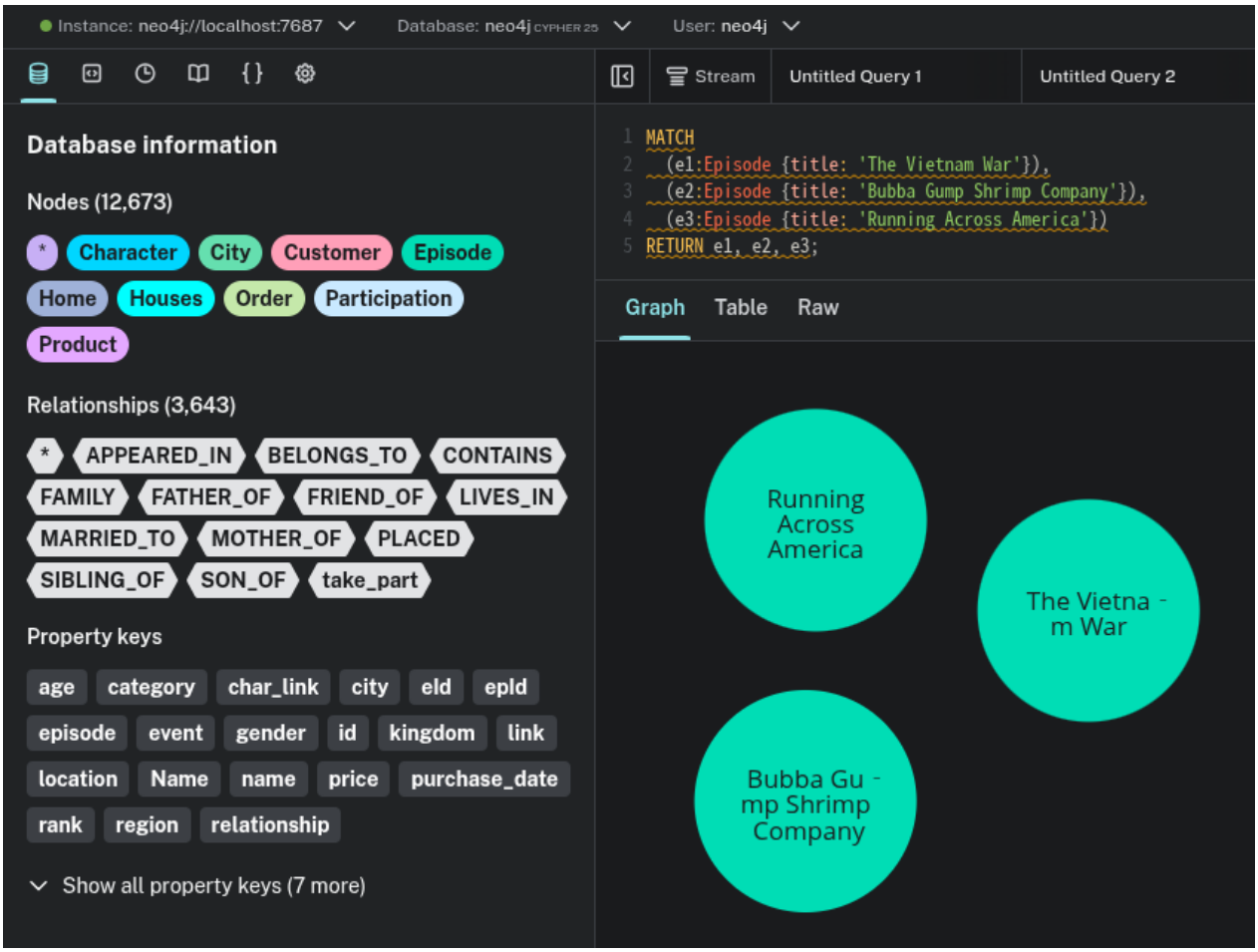


Figure 10: The three Episode nodes displayed in the Neo4j Browser

4 Establishing Links Between Nodes

Nodes alone do not make a graph database useful; the value comes from the relationships. Three groups of relationships are created: family ties, the home of every character, and the episodes each character appeared in.

4.1 Family Relationships

The first group connects the persons to each other. `MATCH` finds the nodes that were created in the previous section and `CREATE` draws the arrows between them.

```
MATCH
(forrest:Character {name: 'Forrest Gump'}),
(mom:Character     {name: 'Mrs. Gump'}),
(jenny:Character   {name: 'Jenny Curran'}),
(junior:Character  {name: 'Forrest Gump Jr.'}),
(bubba:Character   {name: 'Bubba Blue'})
CREATE (mom)-[:MOTHER_OF]->(forrest)
CREATE (forrest)-[:MARRIED_TO]->(jenny)
CREATE (forrest)-[:FATHER_OF]->(junior)
CREATE (jenny)-[:MOTHER_OF]->(junior)
CREATE (forrest)-[:FRIEND_OF]->(bubba)
RETURN 'family relationships created' AS message
```

The relationships that are created are:

- `MOTHER_OF` – from Mrs. Gump to Forrest Gump
- `MARRIED_TO` – from Forrest Gump to Jenny Curran
- `FATHER_OF` – from Forrest Gump to Forrest Gump Jr.
- `MOTHER_OF` – from Jenny Curran to Forrest Gump Jr.
- `FRIEND_OF` – from Forrest Gump to Bubba Blue

The direction of the arrow carries meaning. `FATHER_OF` points from the father to the son, so the same relationship type cannot be used in both directions.

```
with driver.session(database=DATABASE) as session:
    result: Result = session.run(
        """
        MATCH
        (forrest:Character {name: 'Forrest Gump'}),
        (mom:Character     {name: 'Mrs. Gump'}),
        (jenny:Character   {name: 'Jenny Curran'}),
        (junior:Character  {name: 'Forrest Gump Jr.'}),
        (bubba:Character   {name: 'Bubba Blue'})

        CREATE (mom)-[:MOTHER_OF]->(forrest)
        CREATE (forrest)-[:MARRIED_TO]->(jenny)
        CREATE (forrest)-[:FATHER_OF]->(junior)
        CREATE (jenny)-[:MOTHER_OF]->(junior)
        CREATE (forrest)-[:FRIEND_OF]->(bubba)
        RETURN 'family relationships created' AS message
        """
    )

    for record in result:
        print(record["message"])
[18]  ✓ 0.0s
...  family relationships created
```

Figure 11: Creating the family relationships

The returned message `family relationships created` confirms that the whole statement was executed.

The result can also be inspected in the Neo4j Browser. The query below returns every outgoing relationship of the main character that leads to another character. The relationship type is deliberately not fixed in the pattern: `[r]` matches a relationship of any type.

```
MATCH (forrest:Character {name: 'Forrest Gump'})-[r]->(other:Character)
RETURN forrest, r, other;
```

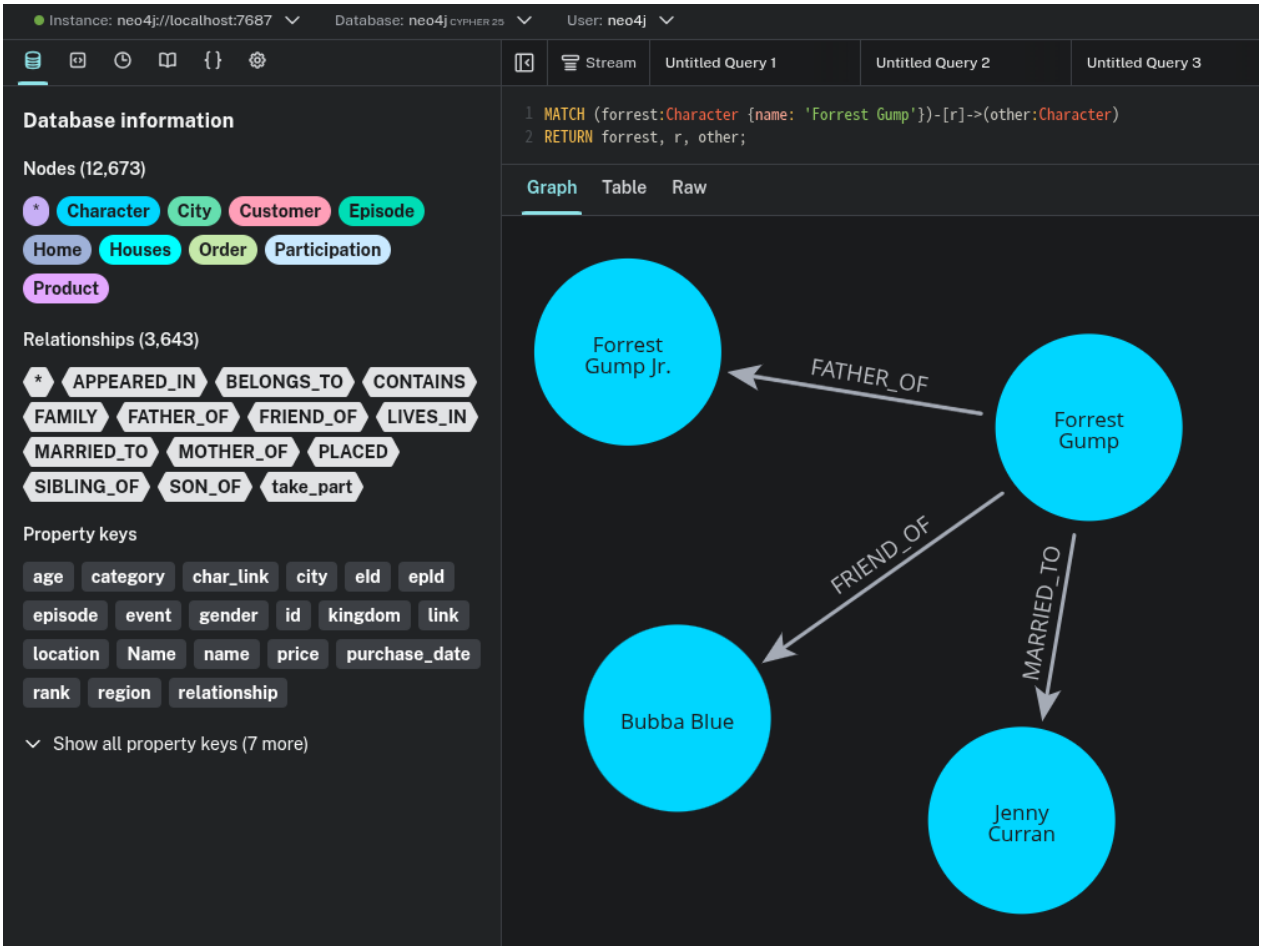


Figure 12: The relationships of the main character in the Neo4j Browser

Three arrows leave the central node: `FATHER_OF` to the son, `MARRIED_TO` to the wife and `FRIEND_OF` to the army friend.

4.2 Home Relationships

Every character is connected to the town where they live through a `LIVES_IN` relationship.

```
MATCH
(forrest:Character {name: 'Forrest Gump'}),
(mom:Character     {name: 'Mrs. Gump'}),
(jenny:Character   {name: 'Jenny Curran'}),
(junior:Character  {name: 'Forrest Gump Jr.'}),
(city:City {name: 'Greenbow'})

CREATE (forrest)-[:LIVES_IN {since: 1944}]->(city)
CREATE (mom)-[:LIVES_IN]->(city)
CREATE (jenny)-[:LIVES_IN]->(city)
```

```
CREATE (junior)-[:LIVES_IN]->(city)
RETURN 'home relationships created' AS message
```

A relationship can hold properties of its own, in the same way as a node. Here the first `LIVES_IN` relationship carries the property `since: 1944`, the year the main character was born in Greenbow. The other three relationships have no properties.



Figure 13: Creating the LIVES_IN relationships to the home town

The result is the message `home relationships created`.

```
MATCH (p:Character)-[r:LIVES_IN]->(c:City {name: 'Greenbow'})
RETURN p, r, c;
```

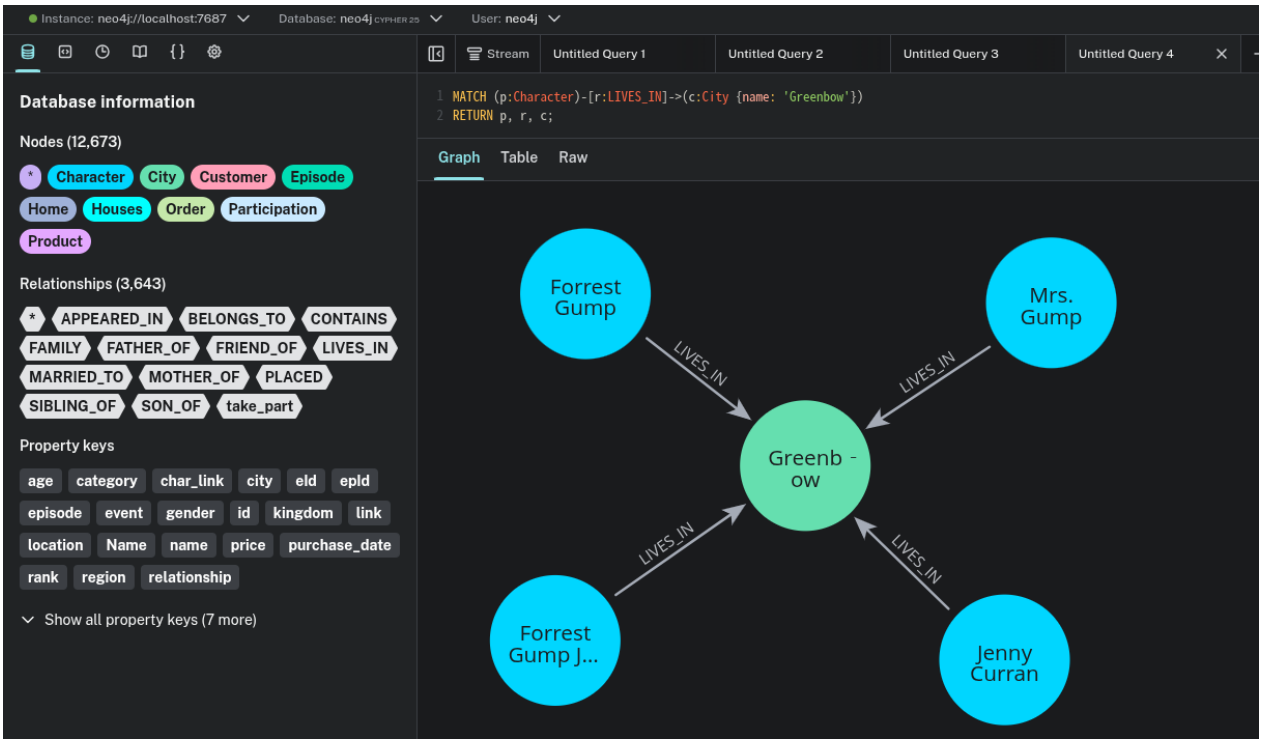


Figure 14: The LIVES_IN relationships pointing to the home town

All four arrows point to the single green `City` node in the middle. This shows the ”many to one” shape of these relationships: many characters, but only one town.

4.3 Episode Relationships

The last group of relationships records which character appears in which episode.

```
MATCH
(forrest:Character {name: 'Forrest Gump'}),
(jenny:Character   {name: 'Jenny Curran'}),
(bubba:Character   {name: 'Bubba Blue'}),
(e1:Episode {title: 'The Vietnam War'}),
(e2:Episode {title: 'Bubba Gump Shrimp Company'}),
(e3:Episode {title: 'Running Across America'})

CREATE (forrest)-[:APPEARED_IN]->(e1)
CREATE (forrest)-[:APPEARED_IN]->(e2)
CREATE (forrest)-[:APPEARED_IN]->(e3)
CREATE (bubba)-[:APPEARED_IN]->(e1)
CREATE (bubba)-[:APPEARED_IN]->(e2)
CREATE (jenny)-[:APPEARED_IN]->(e3)
RETURN 'episode relationships created' AS message
```

The main character appears in all three episodes, Bubba in two of them and Jenny in one.



Figure 15: Creating the APPEARED_IN relationships

The message `episode relationships created` is returned.

The same relationships are shown below in the Neo4j Browser. Here the relationship type is written explicitly in the pattern, so only `APPEARED_IN` is returned.

```
MATCH (forrest:Character {name: 'Forrest Gump'})-[r:APPEARED_IN]->(e:Episode)
RETURN forrest, r, e;
```

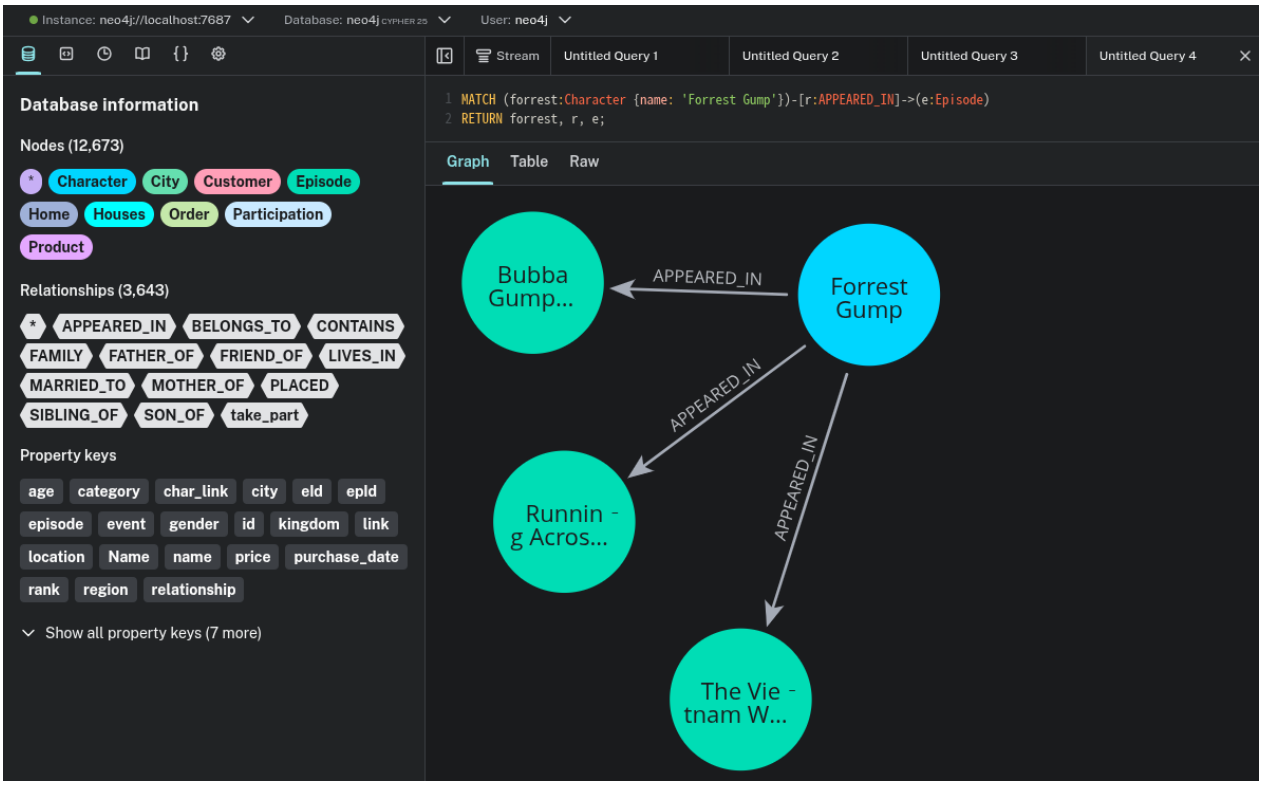


Figure 16: The APPEARED_IN relationships of the main character

The main character is connected to all three episodes through `APPEARED_IN`, which confirms the relationships that were created in the notebook cell above.

5 Reading the Data

5.1 Reading a Single Character

Reading a node back is done with `MATCH` and `RETURN`. The whole node is returned and its properties are read in Python.

```
MATCH (c:Character {name: 'Forrest Gump'})
RETURN c
```

```
    for record in result:
        node = record["c"]
        print("name  :", node["name"])
        print("age   :", node["age"])
        print("state :", node["state"])
        print("role  :", node["role"])
```

The value of `record["c"]` is a Neo4j node object, not a Python dictionary. Its properties are read with the same square bracket syntax, which is why `node["name"]` works.

5.2 Reading the Episodes of a Character

The next query follows the `APPEARED_IN` relationships of one character instead of matching nodes by name.

```
MATCH (c:Character {name: 'Forrest Gump'})-[:APPEARED_IN]->(e:Episode)
RETURN e.title AS episode
```

The pattern starts from the character and walks one hop along the relationship. Only the title of the reached node is returned, aliased as `episode` with the `AS` keyword.



Figure 17: Reading the episodes of the main character

The three episode titles are printed, which confirms that the relationships created in the previous section were stored correctly.

5.3 Reading the Family and the Home

To read the family of a character the arrow is written without a direction. This is necessary because relatives sit on both sides of the arrow: `MOTHER_OF` points from the mother to the son, while `FATHER_OF` points from the father to the son as well, but `MARRIED_TO` points in the other direction for the same pair of people.

```
MATCH (c:Character {name: 'Forrest Gump'})-[r:MOTHER_OF|FATHER_OF|MARRIED_TO]-(
    relative:Character)
RETURN relative.name AS name, relative.role AS role, type(r) AS relation
```

Several relationship types are listed in one pattern, separated by the vertical bar. The function `type(r)` returns the name of the relationship as a string, so the result shows not only who the relative is but also how they are related.

```
MATCH (c:Character)-[:LIVES_IN]->(city:City)
RETURN c.name AS character, city.name AS home
```

The second query in the same cell reads the home of every character.

```python
with driver.session(database=DATABASE) as session:
    result: Result = session.run(
        """
        MATCH (c:Character {name: 'Forrest Gump'})-[r:MOTHER_OF|FATHER_OF|MARRIED_TO]-(relative:Character)
        RETURN relative.name AS name, relative.role AS role, type(r) AS relation
        """
    )

    for record in result:
        print(record["name"], record["role"], record["relation"])

with driver.session(database=DATABASE) as session:
    result: Result = session.run(
        """
        MATCH (c:Character)-[:LIVES_IN]->(city:City)
        RETURN c.name AS character, city.name AS home
        """
    )

    print("--- home ---")
    for record in result:
        print(record["character"], "lives in", record["home"])
```

0.0s

```
Mrs. Gump Mother MOTHER_OF
Jenny Curran Wife MARRIED_TO
Forrest Gump Jr. Son FATHER_OF
--- home ---
Forrest Gump lives in Greenbow
Mrs. Gump lives in Greenbow
Jenny Curran lives in Greenbow
Forrest Gump Jr. lives in Greenbow
```

Figure 18: Reading the family relationships and the home of each character

The output has two parts. The first part lists the mother, the wife and the son together with the relationship type that connects them to the main character. The second part shows that all four characters live in Greenbow.

6 Updating the Data

6.1 Updating the Properties of a Node

An existing property is overwritten and a new one is added with the `SET` clause.

```
MATCH (c:Character {name: 'Forrest Gump'})
SET c.age = 41, c.rank = 'Medal of Honor'
RETURN c
```

The age already exists on the node, so `SET` overwrites its value. The property `rank` does not exist yet, so `SET` adds it to the node. A single `SET` clause can change several properties at once, separated by commas.

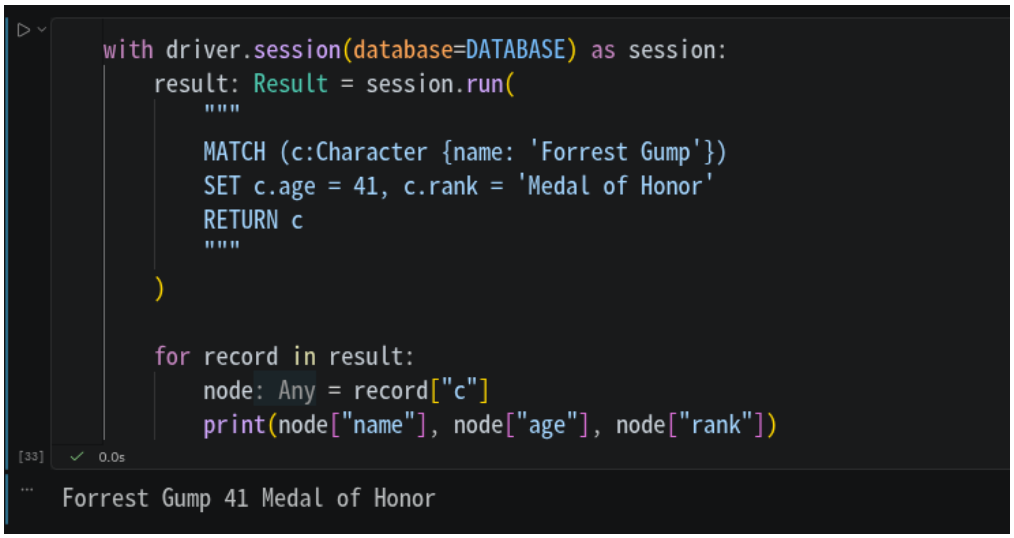


Figure 19: Updating the properties of a Character node

The result `Forrest Gump 41 Medal of Honor` shows the new value of the age and the newly added property.

6.2 Updating the Properties of a Relationship

The same `SET` clause works on a relationship if the relationship is bound to a variable in the `MATCH` pattern. Here the relationship variable is `r`.

```
MATCH (c:Character {name: 'Jenny Curran'})-[r:LIVES_IN]->(city:City)
SET r.since = 1981
RETURN c.name AS character, city.name AS home, r.since AS since
```

This relationship had no `since` property, so `SET` adds it. The value 1981 is used because that is the year the character moved back to Greenbow.

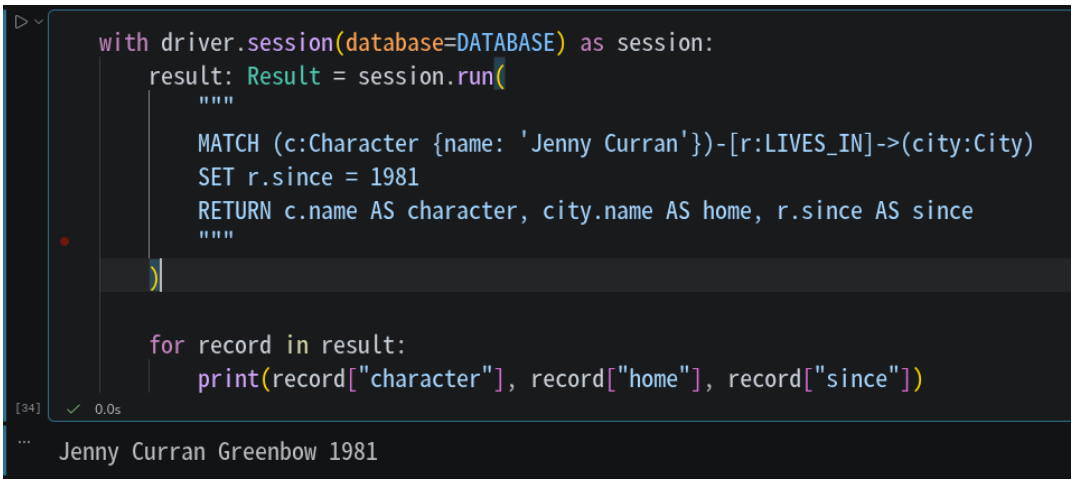


Figure 20: Adding a property to an existing relationship

The output `Jenny Curran Greenbow 1981` confirms that the property was stored on the relationship and not on either of the two nodes.

7 Deleting the Data

To delete a relationship it must first be found with `MATCH` and then removed with the `DELETE` clause.

```
MATCH (forrest:Character {name: 'Forrest Gump'})-[r:FRIEND_OF]->
      (bubba:Character   {name: 'Bubba Blue'})
DELETE r
```

Note that only the relationship variable `r` is passed to `DELETE`. If a node variable were passed instead, the two nodes would be deleted as well. The relationship type `FRIEND_OF` is used here on purpose: it is not part of the family structure, so removing it does not damage the family tree.

The next step is a check that the relationship is really gone. The query counts all the remaining `FRIEND_OF` relationships in the database.

```
    result: Result = session.run("MATCH ()-[r:FRIEND_OF]->() RETURN count(r) AS
    count")
    print(result.single()["count"])
```

The function `result.single()` is used instead of a `for` loop because the query returns exactly one record.

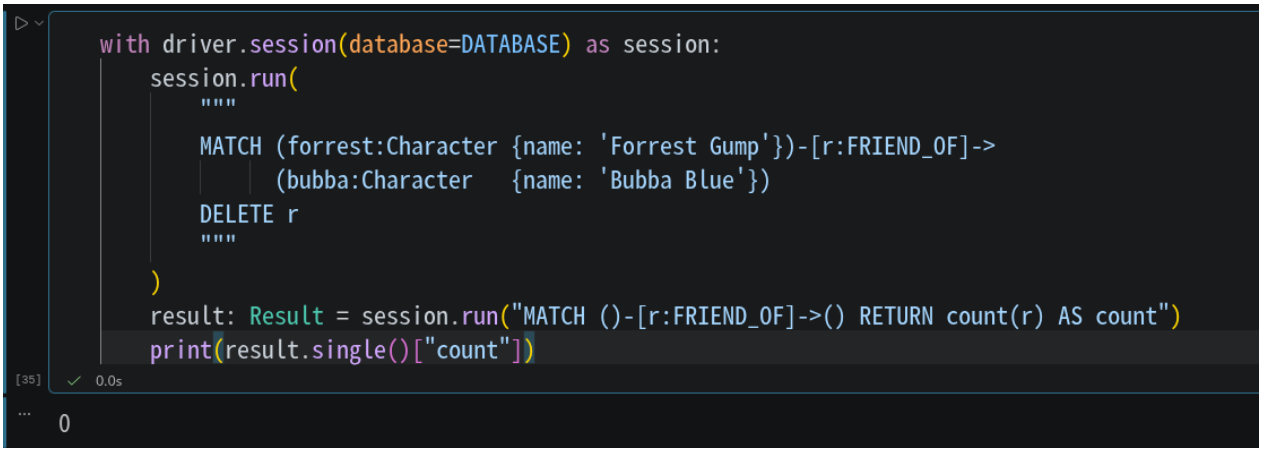


Figure 21: Deleting the FRIEND_OF relationship and checking the result

The printed value is `0`, which proves that the relationship was removed.

8 Building DataFrames from the Data

The last requirement of the laboratory is to turn query results into a pandas DataFrame. The driver makes this easy because a query result can be converted into a list of dictionaries in one step.

8.1 Creating a DataFrame from a Query Result

```
MATCH (c:Character)-[:APPEARED_IN]->(e:Episode)
RETURN c.name AS character, e.title AS episode
ORDER BY episode, character
```

The query returns two columns, one row per appearance. `ORDER BY` sorts the rows first by episode and then by character so that the table is readable.

```
with driver.session(database=DATABASE) as session:
    result: Result = session.run(
        """
        MATCH (c:Character)-[:APPEARED_IN]->(e:Episode)
        RETURN c.name AS character, e.title AS episode
        ORDER BY episode, character
        """
    )

    episodes_df = pd.DataFrame(result.data())

print(episodes_df)
```

The method `result.data()` converts the result into a list of Python dictionaries whose keys are the aliases used in the `RETURN` clause. `pd.DataFrame()` then turns that list into a table, where the keys become the column names. Because whole nodes are not returned here, but only string properties, every column of the DataFrame has a simple type.

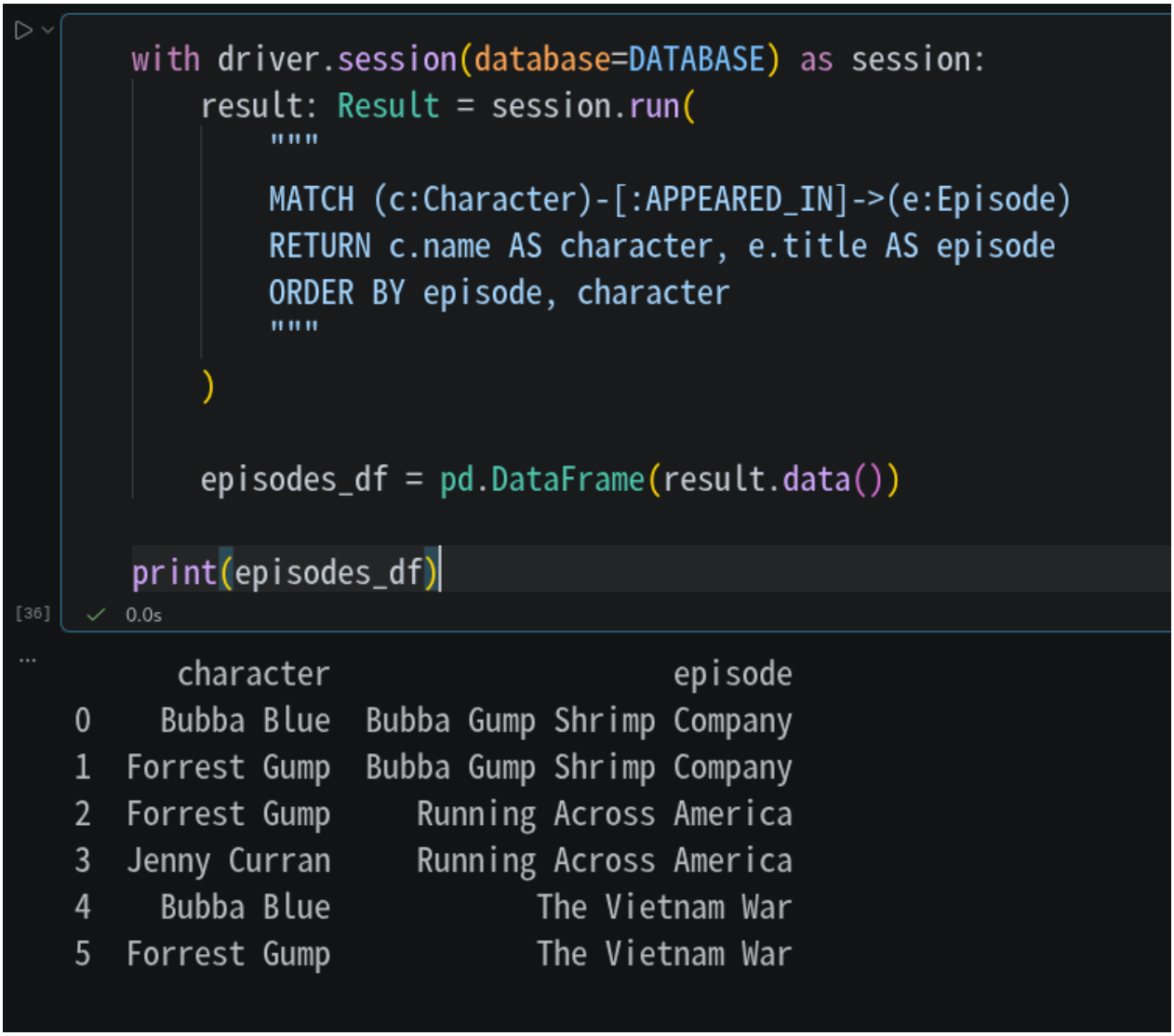


Figure 22: Building a DataFrame from the APPEARED_IN query

The DataFrame has two columns, `character` and `episode`, and six rows one for every relationship of the type `APPEARED_IN`.

8.2 Aggregating the DataFrame and Closing the Connection

Once the data is in a DataFrame, the usual pandas tools can be used on it. The method `value_counts()` counts how many rows belong to each episode.

```
print(episodes_df["episode"].value_counts())
driver.close()
```



Figure 23: Counting the rows per episode and closing the driver

Each of the three episodes has two rows, which means that two characters appear in each of them. This matches the relationships that were created in the `APPEARED_IN` section.

The last line closes the driver. `driver.close()` shuts down the connection pool and frees the resources held by the driver; it should not be omitted, because the driver keeps its connections open until it is closed.

9 Conclusion

In this laboratory the Neo4j database was used from Python with the official `neo4j` driver. All the operations required by the task were performed programmatically:

- the connection was established and verified, and the available databases were listed;
- the nodes for a home town, five characters and three episodes were created with `CREATE`;
- family, home and episode relationships were created with `MATCH` followed by `CREATE`, including a relationship that carries a property of its own;
- the data was read back with `MATCH` and `RETURN`, including a query that walks along a relationship and a query that traverses a relationship in both directions;
- properties of a node and of a relationship were changed with `SET`;

- a relationship was removed with `DELETE` and the result was verified by counting the remaining relationships;
- the query results were converted into pandas DataFrames and aggregated with `value_counts()`.

Each step was also checked in the Neo4j Browser, where the browser screenshots in the previous sections were taken. They confirmed that the nodes and the relationships written from Python are stored in the database exactly as expected.

One practical remark follows from this experiment. The notebook uses `CREATE`, which always creates new nodes and relationships; running the same cells twice therefore produces a duplicate copy of the data, and the duplicates then make queries such as `MATCH (a)-[r]->(b)` return more rows than expected. The `MERGE` clause avoids this by matching an existing pattern before creating a new one, and it is the better choice for any script that may be executed more than once.